



INSTITUTO TECNOLÓGICO DE LAS AMÉRICAS (ITLA)

Escuela de Tecnologías de la Información

Tecnólogo en Desarrollo de Software

TEMA

Automatización del hogar y gestión inteligente de la energía mediante Internet de las Cosas y programación concurrente en tiempo real

TÍTULO DEL PROYECTO

SmartPower RD

Sistema IoT de monitoreo eléctrico con transferencia automática a inversor y notificación remota, construido sobre una arquitectura multitarea en ESP32

DATOS DE LA ASIGNATURA Y DEL SUSTENTANTE	
Asignatura	Inteligencia Artificial e Internet de las Cosas
Código / Sección	2026-C-2
Facilitador	Luis Bessewell Feliz
Sustentante	Cristian Carrera
Matrícula	2024-1932
Correo institucional	20241932@itla.edu.do
Actividad	Investigación previa y caso práctico de laboratorio
Fecha de entrega	6 de agosto de 2026
Lugar	Santo Domingo, República Dominicana

Trabajo presentado como requisito parcial de la asignatura. Todo el código fuente, los diagramas y las mediciones incluidas en este informe fueron elaborados por el sustentante y son reproducibles a partir de los enlaces del Anexo A.

Índice de contenido

Índice de contenido	3
1. Introducción	5
2. Objetivos	5
2.1 Objetivo general.....	5
2.2 Objetivos específicos.....	5
3. Planteamiento del problema y justificación	6
3.1 El contexto dominicano	6
3.2 Por qué un microcontrolador y no una aplicación.....	6
3.3 Por qué el problema es de programación concurrente.....	6
4. Marco conceptual: programación en hilos y tareas	8
4.1 Proceso, hilo y tarea: tres cosas distintas	8
4.2 Concurrencia no es paralelismo.....	8
4.3 Programación distribuida.....	9
4.4 Mecanismos de sincronización	9
4.5 Los tres errores clásicos, y cómo se evitan aquí	10
Condición de carrera	10
Interbloqueo (deadlock)	10
Inversión de prioridad.....	10
5. Desarrollo del caso práctico: SmartPower RD	12
5.1 Descripción funcional.....	12
5.2 Arquitectura del sistema.....	12
5.3 Componentes de hardware y su función	12
5.4 Arquitectura de tareas: la decisión central del diseño	13
5.5 Flujo de datos y de sincronización	13
5.6 Fragmento representativo del código	14
5.7 Extensión con inteligencia artificial	15
5.8 Implementación en Arduino Cloud	15
El problema que plantea ArduinoCloud.update().....	15
Seguridad entre hilos de las variables de nube	16
Variables de nube y tablero	16
6. Revisión del diseño a partir de la retroalimentación	17
6.1 ¿Una fase o dos fases?	17
Fase perdida.....	17

Neutro perdido	17
6.2 ¿Se protege el inversor o se protegen los electrodomésticos?	18
6.3 La alimentación del propio monitor	18
7. Evidencias demostrables de programación concurrente	20
7.1 Ejemplo 1 — El problema: programación secuencial bloqueante.....	20
7.2 Ejemplo 2 — Multitarea cooperativa con millis()	20
7.3 Ejemplo 3 — Hilos reales, expulsión y dos núcleos	21
7.4 Ejemplo 4 — La condición de carrera y su cura	21
7.5 Ejemplo 5 — Programación distribuida con MQTT.....	21
7.6 Ejemplo 6 — Concurrencia y paralelismo, medidos con cronómetro	22
8. Presupuesto y cotización	23
8.1 Escenario A — Prototipo académico simulado.....	23
8.2 Escenario B — Implementación física en una vivienda	23
8.2.1 Materiales y componentes	23
8.2.2 Logística, impuestos y servicios	24
8.2.3 Costos recurrentes	24
8.3 Análisis de retorno de la inversión.....	25
9. Cronograma de implementación	26
10. Conclusión.....	26
10.1 Recomendaciones	27
11. Fuentes de investigación y bibliografía.....	28
11.1 Libros y artículos académicos	28
11.2 Documentación técnica y normas	28
11.3 Conferencias y recursos en línea	28
11.4 Fuentes de datos económicos	28
Anexo A. Enlaces de acceso al código y a los videos	29
A.1 Código fuente	29
A.2 Material audiovisual de referencia.....	29
A.3 Qué se entrega.....	31
Cómo reproducir cada demostración	31

1. Introducción

En la República Dominicana la energía eléctrica no es un servicio que se dé por sentado. Las interrupciones del suministro, las variaciones bruscas de voltaje y el uso masivo de inversores y bancos de baterías forman parte de la rutina doméstica de millones de hogares. Sin embargo, la mayoría de esas instalaciones opera a ciegas: el usuario se entera de que hubo un apagón porque se apagó la luz, y se entera de que un electrodoméstico sufrió una sobretensión porque dejó de funcionar.

Este trabajo parte de una pregunta muy concreta que me hice como estudiante y como usuario: ¿qué me gustaría construir con un Arduino que resolviera un problema real de mi propia casa? La respuesta fue SmartPower RD, un sistema de Internet de las Cosas que mide continuamente el voltaje y la corriente de la acometida eléctrica, decide por sí mismo cuándo transferir la carga al inversor, protege los equipos ante voltajes peligrosos y notifica al usuario en su teléfono, esté donde esté.

Ahora bien, el interés académico del proyecto no está solamente en el circuito. Un sistema como este obliga a enfrentar un problema de programación que rara vez aparece en los ejercicios introductorios: hay que hacer cuatro cosas distintas al mismo tiempo, con exigencias de tiempo incompatibles entre sí. Medir el voltaje no puede esperar; publicar en la nube puede tardar segundos. Si ambas responsabilidades se escriben en el mismo bucle secuencial, el sistema falla justo cuando más se le necesita.

Por esa razón, la segunda mitad de este informe está dedicada a investigar y demostrar los conceptos de programación en hilos y tareas, y su aplicación a la programación concurrente, paralela y distribuida. No se trata de un anexo teórico: cada concepto se acompaña de un programa ejecutable y de una medición reproducible que prueba, con números, la diferencia entre hacerlo bien y hacerlo mal.

El documento se organiza así: primero se plantea el problema y los objetivos; luego se desarrolla el marco conceptual de la concurrencia; a continuación se detalla la solución técnica de SmartPower RD con su arquitectura de tareas; después se presentan las seis demostraciones prácticas con sus evidencias; y finalmente se ofrece el presupuesto detallado, el cronograma, las conclusiones y las fuentes consultadas.

2. Objetivos

2.1 Objetivo general

Diseñar, programar y documentar un sistema IoT de monitoreo y protección eléctrica para viviendas dominicanas, basado en una arquitectura multitarea sobre ESP32, que demuestre de forma verificable la aplicación de la programación concurrente, paralela y distribuida a un problema real.

2.2 Objetivos específicos

- Investigar y delimitar con precisión los conceptos de proceso, hilo, tarea, concurrencia, paralelismo y sistema distribuido, distinguiendo lo que a menudo se confunde en la práctica.

- Construir una arquitectura de cinco tareas de FreeRTOS con prioridades y períodos diferenciados, distribuidas entre los dos núcleos del ESP32, justificando cada asignación.
- Aplicar correctamente los tres mecanismos de sincronización fundamentales —colas de mensajes, mutex y semáforos— y explicar qué problema concreto resuelve cada uno dentro del proyecto.
- Demostrar experimentalmente, con mediciones repetibles, el costo de programar mal la concurrencia: bloqueo del sistema por `delay()`, condiciones de carrera y pérdida silenciosa de datos.
- Extender el sistema a una topología distribuida con varios nodos y un concentrador, comunicados por MQTT, incorporando tolerancia a fallos parciales mediante testamento (LWT) y mensajes retenidos.
- Elaborar un presupuesto detallado y un cronograma de implementación que permitan evaluar la viabilidad económica del proyecto en el mercado dominicano.
- Publicar todo el código fuente y los videos de las demostraciones en enlaces de acceso público que permitan al facilitador validar cada afirmación de este informe.

3. Planteamiento del problema y justificación

3.1 El contexto dominicano

El problema que este proyecto ataca tiene tres caras, y todas cuestan dinero al usuario final:

- **Interrupciones del suministro.** El hogar promedio con inversor no sabe cuántos cortes tuvo, cuánto duraron, ni cuánta energía consumió de la batería. Sin ese dato es imposible dimensionar correctamente un banco de baterías o justificar la inversión en paneles solares.
- **Variaciones de voltaje.** Una subtensión sostenida es lo que quema los compresores de neveras y aires acondicionados, porque el motor intenta arrancar con corriente excesiva. El daño no es inmediato ni visible: se acumula durante meses y aparece como una falla repentina y costosa.
- **Ausencia total de visibilidad.** El usuario recibe una factura mensual agregada. No sabe qué circuito consume qué, ni a qué hora, ni si el aire acondicionado quedó encendido cuando salió de la casa.

3.2 Por qué un microcontrolador y no una aplicación

Podría pensarse que este problema se resuelve con software en la nube. No es así: el sistema tiene que actuar sobre el mundo físico —abrir y cerrar un contactor de transferencia— y tiene que hacerlo en menos de medio segundo, incluso cuando no hay internet. Un servicio en la nube que dependa de la conectividad falla precisamente en el escenario para el que fue diseñado, porque un apagón también tumba el router. La inteligencia debe residir en el borde de la red (edge computing), y la nube debe usarse para lo que sí es buena: histórico, análisis y notificación.

3.3 Por qué el problema es de programación concurrente

Aquí está el núcleo académico del trabajo. El sistema debe cumplir simultáneamente cuatro compromisos con plazos que difieren en dos órdenes de magnitud:

Tabla 1. Compromisos temporales del sistema y consecuencia de incumplirlos.

Responsabilidad	Plazo	Qué ocurre si se incumple
Muestreo eléctrico	50 ms	Se pierde el evento de sobretensión que debía detectarse.
Lógica de protección	≤ 500 ms	El electrodoméstico queda expuesto al voltaje peligroso.
Refresco de pantalla	500 ms	La interfaz se percibe congelada; el usuario desconfía.
Publicación en la nube	5 s	Se retrasa la notificación; no es crítico para la seguridad.

Nota: el bus I2C de la pantalla tarda cerca de 30 ms por refresco completo, y un intento de reconexión WiFi puede tardar hasta 8 segundos. En un diseño secuencial, esos 8 segundos dejan al sistema ciego.

La aritmética es implacable: si un solo intento de reconexión WiFi bloquea el programa durante ocho segundos, el sistema deja de mirar el voltaje durante ciento sesenta ciclos de muestreo consecutivos. Ningún ajuste de constantes arregla eso. Lo que hay que cambiar es el modelo de ejecución, y ese es exactamente el territorio de la programación concurrente.

4. Marco conceptual: programación en hilos y tareas

Esta sección responde al segundo componente de la asignación. Se definen los conceptos en sentido general, tal como se estudian en sistemas operativos, y a continuación se traducen al vocabulario específico de los microcontroladores, donde los mismos problemas aparecen con otros nombres.

4.1 Proceso, hilo y tarea: tres cosas distintas

La confusión más común en este tema nace de usar las tres palabras como sinónimos. No lo son, y la diferencia es la memoria.

Tabla 2. Diferencias fundamentales entre proceso, hilo y tarea.

Concepto	Qué posee	Costo de cambio	Riesgo característico
Proceso	Su propio espacio de memoria virtual, aislado del resto	Alto (cambio de tablas de páginas)	Comunicarse es caro: requiere IPC, sockets o memoria compartida explícita
Hilo	Solo su pila y sus registros; la memoria la comparte con sus hermanos	Bajo	Condiciones de carrera: todos escriben sobre los mismos datos
Tarea (RTOS)	Su pila y su prioridad; no hay memoria virtual que aislar	Muy bajo (microsegundos)	Igual que el hilo, más el desbordamiento de pila silencioso

En un microcontrolador no existe unidad de gestión de memoria, así que no existen procesos en el sentido estricto. Lo que FreeRTOS llama tarea es conceptualmente un hilo: comparte todo el espacio de memoria con las demás tareas y solo posee su propia pila. Esto tiene una consecuencia práctica importante: en Arduino, cualquier variable global es, por definición, una variable compartida entre todas las tareas, y por lo tanto un candidato a condición de carrera.

El componente que decide qué tarea corre en cada instante es el planificador (scheduler). Su política define dos familias de sistemas:

- **Cooperativo:** una tarea corre hasta que decide voluntariamente ceder el control. Es simple y predecible, pero una sola tarea mal escrita congela todo el sistema. El bucle `loop()` clásico de Arduino con máquinas de estado basadas en `millis()` pertenece a esta familia.
- **Con expulsión (preemptive):** el planificador interrumpe a una tarea sin pedirle permiso, ya sea porque venció su turno o porque una tarea de mayor prioridad quedó lista para ejecutarse. Es lo que hacen FreeRTOS, Linux y Windows. Da garantías de tiempo real, a cambio de exponer al programador a todos los problemas de sincronización.

4.2 Concurrencia no es paralelismo

Esta distinción es la que más consecuencias prácticas tiene, y conviene fijarla con una frase que se ha vuelto clásica en la disciplina:

«La concurrencia consiste en lidiar con muchas cosas a la vez; el paralelismo consiste en hacer muchas cosas a la vez.»

La concurrencia es una propiedad del diseño del programa: describe tareas cuyo tiempo de vida se solapa y que progresan de forma intercalada. Puede lograrse perfectamente sobre un solo núcleo. El paralelismo, en cambio, es una propiedad del hardware que lo ejecuta: exige dos o más unidades de cómputo físicas ejecutando instrucciones en el mismo instante.

Un programa puede ser concurrente sin ser paralelo (el Ejemplo 2 de este trabajo, sobre un Arduino UNO de un solo núcleo), y el mismo programa concurrente se vuelve paralelo al ejecutarse sobre un ESP32 de dos núcleos (Ejemplo 3). El diseño no cambia; cambia la máquina.

La confusión tiene un costo medible, y este trabajo lo cuantifica en la sección 6: usar hilos para acelerar un cálculo intensivo en CPU dentro de CPython produce una aceleración de 0.99×, es decir, ninguna.

4.3 Programación distribuida

Se da un tercer salto cuando las unidades de ejecución dejan de compartir memoria y pasan a vivir en máquinas distintas conectadas por una red. Aquí desaparecen tres supuestos que hasta ahora se daban por garantizados:

- No hay memoria común. La única forma de coordinarse es intercambiar mensajes, que pueden perderse, duplicarse o llegar desordenados.
- No hay reloj común. Cada nodo tiene su propio `millis()`, que arrancó en un momento distinto. Ordenar eventos globalmente requiere números de secuencia o relojes lógicos.
- Existe el fallo parcial. Un nodo puede morir mientras los demás siguen funcionando, y lo peor es que los demás no se enteran: desde fuera, un nodo caído y un nodo lento se ven exactamente igual.

El Ejemplo 5 de este trabajo enfrenta los tres: usa números de secuencia por nodo para ordenar, mensajes retenidos para que un suscriptor nuevo conozca el último estado, y el mecanismo de testamento (Last Will and Testament) de MQTT para que el broker anuncie automáticamente la muerte de un nodo que dejó de responder.

4.4 Mecanismos de sincronización

Cuando varias tareas comparten datos hacen falta herramientas para coordinarlas. Estas son las cuatro que se utilizan en SmartPower RD, con el problema exacto que resuelve cada una:

Tabla 3. Mecanismos de sincronización empleados en el proyecto.

Mecanismo	Para qué sirve	Dónde se usa en SmartPower RD
Cola (queue)	Pasar datos entre tareas por copia, sin variables compartidas	La tarea de muestreo produce estructuras Muestra y la tarea de control las consume. La cola ya es segura entre tareas, así que no hace falta ningún candado adicional.
Mutex	Garantizar que solo una tarea a la vez toque un dato compartido	Protege la estructura EstadoSistema, que la tarea de control escribe mientras la pantalla y la red la leen.

Mecanismo	Para qué sirve	Dónde se usa en SmartPower RD
Semáforo binario	Señalizar que ocurrió un evento, para despertar a una tarea dormida	La tarea de control lo entrega cuando cambia el estado eléctrico; la tarea de red duerme sobre él y publica la alerta al instante, sin encuestar.
Semáforo contador	Contar recursos disponibles o eventos pendientes; construir barreras	En el Ejemplo 4 se usa como barrera: el coordinador no continúa hasta recoger las dos señales de «ya terminé».
Sección crítica (portMUX)	Bloquear muy brevemente la expulsión y las interrupciones	Protege el contador que modifica la rutina de interrupción del botón de silencio, que puede ejecutarse en el otro núcleo.

Una regla de diseño atraviesa toda la implementación y conviene enunciarla explícitamente: la sección crítica debe ser lo más corta posible. En la tarea de control, el mutex se toma para copiar seis campos y se libera de inmediato; los cálculos, las escrituras a los pines y las impresiones por el puerto serie quedan deliberadamente fuera. Una sección crítica larga anula, en la práctica, las ventajas de haber separado el sistema en tareas.

4.5 Los tres errores clásicos, y cómo se evitan aquí

Condición de carrera

Ocurre cuando el resultado depende del orden imprevisible en que se intercalen las tareas. Su forma más frecuente es la actualización perdida: la instrucción total $+= 1$ no es atómica, sino tres pasos de máquina (leer, sumar, escribir), y si dos tareas se intercalan entre el primero y el tercero, ambas leen el mismo valor viejo y una de las dos sumas desaparece sin dejar rastro. El Ejemplo 4 lo reproduce en el ESP32 y el Ejemplo 6 en Python; en este último se pierde el 74.9 % de las operaciones.

Interbloqueo (deadlock)

Dos tareas se esperan mutuamente para siempre: A tiene el candado 1 y pide el 2, mientras B tiene el 2 y pide el 1. Las dos defensas aplicadas en este proyecto son estándar y baratas: primera, ordenar siempre la toma de candados según una jerarquía fija; segunda, no usar nunca `portMAX_DELAY` en el código de producción, sino un vencimiento explícito —en SmartPower RD, `xSemaphoreTake(mutexEstado, pdMS_TO_TICKS(100))`—, de modo que un candado atascado degrade el servicio en lugar de congelar el sistema.

Inversión de prioridad

Una tarea de alta prioridad queda bloqueada esperando un candado que retiene una tarea de baja prioridad, y esta última es a su vez desplazada por tareas de prioridad media. El resultado es que la tarea más urgente del sistema queda, en la práctica, con la prioridad de la menos urgente. El mutex de FreeRTOS lo mitiga mediante herencia de prioridad: mientras retiene el candado, la tarea de baja prioridad es promovida temporalmente a la prioridad de la tarea que lo espera. Esta es la razón técnica por la que en FreeRTOS se debe usar `xSemaphoreCreateMutex()` y no un semáforo binario para proteger datos: el semáforo binario no implementa herencia.

Este fallo no es teórico: en 1997 el vehículo explorador Mars Pathfinder sufrió reinicios repetidos en la superficie de Marte a causa de una inversión de prioridad en VxWorks. El diagnóstico y la corrección se realizaron de forma remota, activando la herencia de prioridad en el mutex responsable.

5. Desarrollo del caso práctico: SmartPower RD

5.1 Descripción funcional

SmartPower RD se instala en el tablero principal de la vivienda, entre la acometida y el inversor. De forma permanente mide el voltaje y la corriente de la línea y, en función de lo que encuentra, ejecuta cuatro comportamientos:

- Con la red presente y dentro del rango seguro (100 V a 135 V), mantiene la carga conectada a la red y publica telemetría cada cinco segundos.
- Con la red presente pero fuera de rango, transfiere la carga al inversor para proteger los equipos y enciende la alarma de voltaje anormal.
- Ante un apagón (voltaje inferior a 40 V), declara el corte, espera 200 ms para descartar un parpadeo momentáneo, confirma la transferencia al inversor y publica una alerta inmediata.
- En todo momento acumula la energía consumida, cuenta los cortes y registra el tiempo total en batería, datos que permiten dimensionar correctamente el banco de baterías.

5.2 Arquitectura del sistema

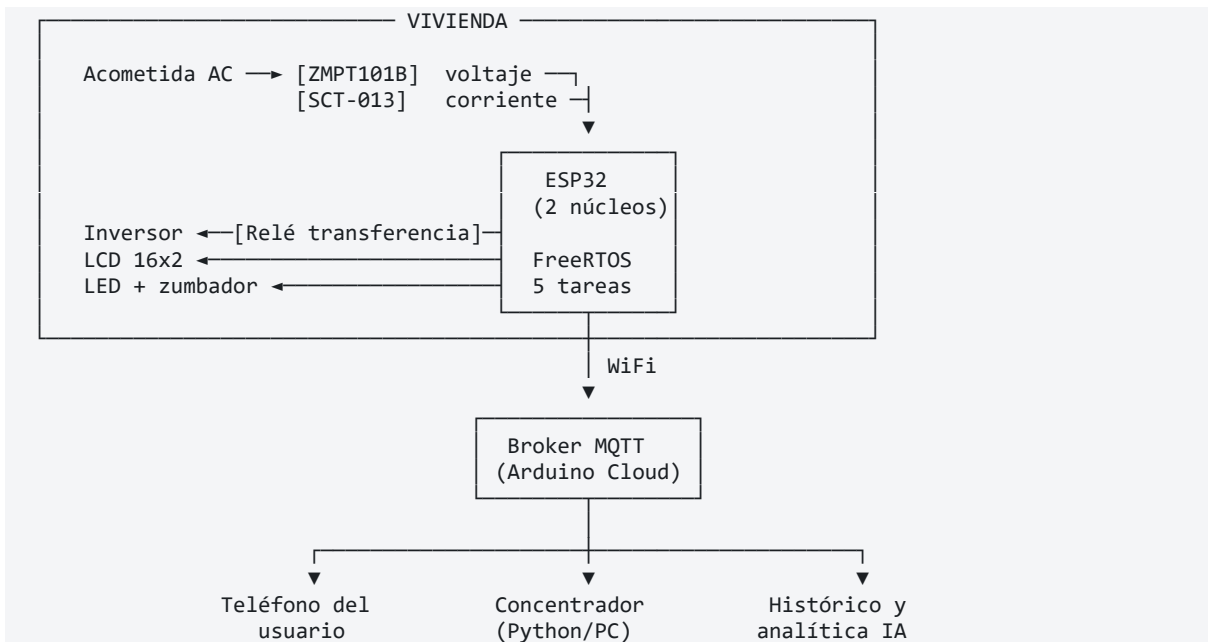


Figura 1. Arquitectura general de SmartPower RD.

5.3 Componentes de hardware y su función

Tabla 4. Componentes principales y justificación de su selección.

Componente	Modelo	Por qué se eligió
Microcontrolador	ESP32 DevKit V1	Dos núcleos (permite paralelismo real), WiFi y Bluetooth integrados, ADC de 12 bits y FreeRTOS incluido en el propio SDK de Arduino.

Componente	Modelo	Por qué se eligió
Sensor de voltaje	ZMPT101B	Transformador de precisión con aislamiento galvánico: separa físicamente los 120 V AC del circuito de 3.3 V del microcontrolador.
Sensor de corriente	SCT-013-030	Pinza de efecto Hall no invasiva: se instala abrazando el conductor, sin cortar el cable ni interrumpir el servicio.
Actuador	Relé/contactador 30 A	Debe soportar la corriente total de la vivienda y una vida útil de decenas de miles de conmutaciones.
Interfaz local	LCD 16x2 I2C	Garantiza que el sistema siga siendo útil aunque no haya internet, que es justamente el escenario de un apagón.
Alimentación	Hi-Link HLK-PM01	Fuente conmutada compacta de 120 V AC a 5 V DC, diseñada para montarse dentro del mismo gabinete.

5.4 Arquitectura de tareas: la decisión central del diseño

Aquí es donde el proyecto deja de ser un ejercicio de electrónica y se convierte en uno de ingeniería de software. Cada responsabilidad se aisló en su propia tarea de FreeRTOS, con prioridad, período y núcleo asignados de forma deliberada.

Tabla 5. Arquitectura de tareas de SmartPower RD.

Tarea	Núcleo	Prior.	Período	Responsabilidad y justificación
Muestreo	1	3	50 ms	Única tarea que toca el ADC. Prioridad máxima porque es la que no puede perder plazos. Usa <code>vTaskDelayUntil</code> para un período estable sin deriva acumulativa.
Control	1	2	por evento	Máquina de estados y actuación sobre el relé. Se bloquea en la cola: mientras no hay datos no consume CPU alguna.
Pantalla	1	1	500 ms	El bus I2C es lento (~30 ms). Prioridad baja para que el planificador la expulse sin contemplaciones cuando el control necesite la CPU.
Red	0	1	5 s o evento	Aislada en el núcleo 0, donde reside el stack de WiFi. Un bloqueo de 8 s reconectando jamás retrasa la detección del apagón.
Diagnóstico	0	0	10 s	Reporta la pila libre de cada tarea (high water mark) y el heap disponible. Permite dimensionar las pilas con datos en lugar de adivinar.

La separación por núcleos merece una explicación adicional. El núcleo 0 del ESP32 es donde Espressif ejecuta el stack de WiFi y Bluetooth. Anclar allí la tarea de red no es una preferencia estética: evita que las interrupciones del radio compitan con las tareas de tiempo real del núcleo 1. Es una decisión de arquitectura que aprovecha el hardware en lugar de ignorarlo.

5.5 Flujo de datos y de sincronización

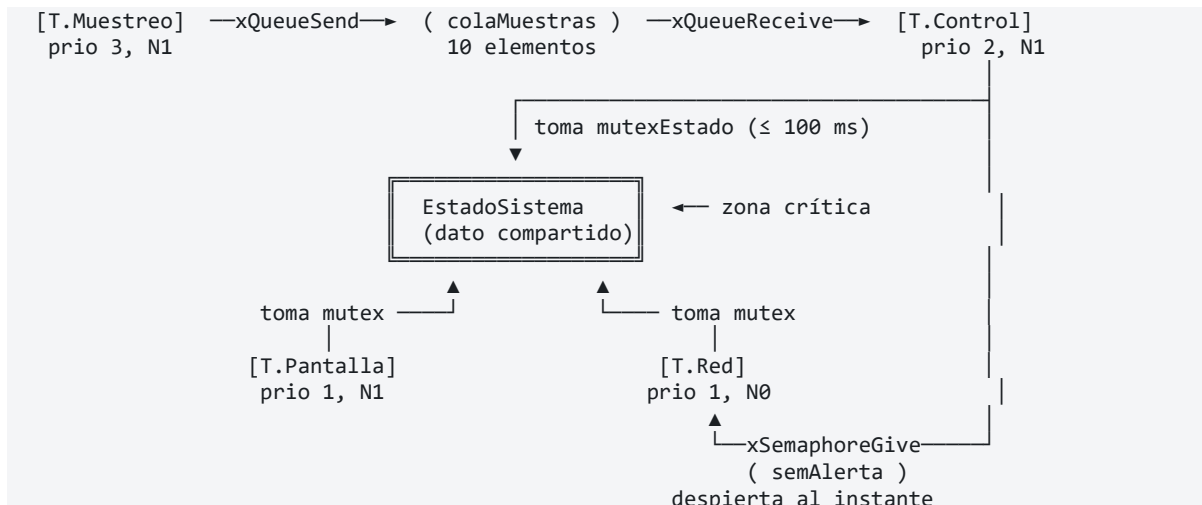


Figura 2. Flujo de datos entre tareas. La cola transporta muestras; el mutex protege el estado compartido; el semáforo señala eventos.

Obsérvese el detalle de diseño más importante de todo el programa: la tarea de control nunca llama a MQTT. Publicar puede tardar segundos, y la tarea de control es crítica. Lo único que hace es entregar el semáforo —una operación de microsegundos— y seguir trabajando. La tarea de red, que dormía sobre ese semáforo, despierta y se encarga. Esta separación entre «detectar» y «comunicar» es la que permite que el sistema mantenga sus garantías de tiempo real aunque la red esté caída.

5.6 Fragmento representativo del código

El siguiente extracto de la tarea de control condensa las tres ideas centrales del diseño. El código fuente completo está disponible en los enlaces del Anexo A.

```

// Bloqueo hasta que llegue una muestra. Mientras espera, esta tarea no
// consume nada de CPU: el planificador se la entrega a otras.
if (xQueueReceive(colaMuestras, &m, portMAX_DELAY) != pdTRUE) continue;

// ... máquina de estados y actuación sobre el relé ...

// Zona crítica: se toma el mutex el MENOR tiempo posible. Todo el
// trabajo pesado queda deliberadamente fuera del candado.
if (xSemaphoreTake(mutexEstado, pdMS_TO_TICKS(100)) == pdTRUE) {
    estado.voltaje = m.voltaje;
    estado.corriente = m.corriente;
    estado.estado = nuevoEstado;
    bool sonar = (nuevoEstado != RED_NORMAL) && !estado.alarmaSilenciada;
    xSemaphoreGive(mutexEstado); // se libera ANTES de tocar el zumbador

    if (sonar) tone(PIN_ZUMBADOR, 2000, 120);
    else      noTone(PIN_ZUMBADOR);
}

// No se llama a MQTT desde aquí: publicar puede tardar segundos y esta
// tarea es crítica. Solo se levanta la bandera y se sigue trabajando.
if (nuevoEstado != estadoAnterior) {
    xSemaphoreGive(semAlerta);
    estadoAnterior = nuevoEstado;
}

```

Figura 3. Extracto de tareaControl() en SmartPowerRD.ino.

5.7 Extensión con inteligencia artificial

El proyecto está preparado para una segunda fase que conecta directamente con la otra mitad de la asignatura. Los datos que SmartPower RD acumula —voltaje, corriente, potencia y duración de los cortes, muestreados cada cinco segundos— constituyen exactamente la serie temporal que necesita un modelo predictivo. Tres aplicaciones son viables sin hardware adicional:

- **Desagregación de carga (NILM).** A partir de la firma de consumo del circuito principal es posible identificar qué electrodoméstico se encendió, sin instalar un sensor por aparato.
- **Predicción de la autonomía de la batería.** Un modelo de regresión entrenado con el histórico de cortes y consumo puede estimar cuántas horas de respaldo quedan, en lugar de mostrar un porcentaje que no significa nada para el usuario.
- **Detección de anomalías.** Un aumento gradual de la corriente de arranque del compresor de la nevera es el precursor típico de su falla. Un detector de anomalías sobre esa serie permite avisar semanas antes de que ocurra.

El entrenamiento de estos modelos es trabajo intensivo en CPU y se ejecuta en el concentrador o en la nube, con procesos paralelos —no con hilos, por las razones que demuestra el Ejemplo 6—. La inferencia, una vez cuantizado el modelo, sí puede correr en el propio ESP32 con TensorFlow Lite for Microcontrollers, en una sexta tarea de baja prioridad.

5.8 Implementación en Arduino Cloud

El enunciado señala que sería inspirador programar la solución utilizando Arduino Cloud de forma virtual, o bien un Arduino físico con el Arduino IDE. El proyecto atiende esa indicación con una segunda versión completa del firmware, alojada en la carpeta SmartPowerRD_ArduinoCloud, que sustituye el broker MQTT propio por la plataforma oficial de Arduino.

Lo relevante de esta versión no es que funcione, sino cuánto hubo que cambiar para que funcionara: de las cinco tareas del sistema, se reescribió una sola. El muestreo, el control, la pantalla y el diagnóstico quedaron intactos, byte por byte. Esa es la prueba práctica de que la separación de responsabilidades en tareas estaba bien planteada; en un programa secuencial, cambiar el transporte habría obligado a reescribirlo entero.

El problema que plantea `ArduinoCloud.update()`

La biblioteca `ArduinoloTCloud` exige que se invoque `ArduinoCloud.update()` con frecuencia. Esa única llamada negocia TLS con los servidores de Arduino, reconecta el WiFi si se cayó y sincroniza todas las variables de nube, de modo que puede bloquearse varios segundos. Todos los ejemplos oficiales la colocan dentro de `loop()`.

En este proyecto eso sería un error de diseño grave: `loop()` compartiría hilo con la rutina que decide si transferir la carga al inversor, y durante cada reconexión el sistema dejaría de vigilar el voltaje, precisamente en el escenario para el que fue construido. La solución es la misma que en la versión MQTT y refuerza toda la tesis del trabajo: la llamada lenta vive en su propia tarea, anclada al núcleo 0 junto al stack de WiFi, con prioridad 1.

Seguridad entre hilos de las variables de nube

La biblioteca `ArduinoloTCloud` no es segura entre hilos. Por eso se aplica una regla estricta, idéntica en espíritu a la del `mutex` de la sección 5.5: las variables de nube las escribe únicamente la tarea de nube, y sus valores salen de una copia del estado compartido tomada bajo `mutex`. Ninguna otra tarea toca una variable de nube, de modo que nunca hay dos hilos dentro de la biblioteca al mismo tiempo. Los propios `callbacks` del tablero —los que se disparan cuando el usuario mueve un interruptor en el teléfono— se ejecutan dentro de esa misma tarea y toman el `mutex` como cualquier otro código: un `callback` de nube no es código privilegiado.

Variables de nube y tablero

Tabla 6. Variables del Thing en Arduino Cloud.

Variable	Tipo	Permiso	Actualización	Widget del tablero
voltaje	float	Solo lectura	Al cambiar (Δ 0.5)	Medidor 0–260 V
corriente	float	Solo lectura	Al cambiar (Δ 0.1)	Medidor 0–30 A
potencia	float	Solo lectura	Al cambiar (Δ 10)	Valor en W
energiaWh	float	Solo lectura	Cada 30 s	Gráfica histórica
estadoRed	String	Solo lectura	Al cambiar	Estado en texto
cortesDetectados	int	Solo lectura	Cada 30 s	Contador
minutosEnInversor	int	Solo lectura	Cada 30 s	Contador
silenciarAlarma	bool	Lectura y escritura	Al cambiar	Interruptor
umbralMinimo	float	Lectura y escritura	Al cambiar	Deslizador 80–120 V
umbralMaximo	float	Lectura y escritura	Al cambiar	Deslizador 125–150 V

La política de actualización de cada variable no es un detalle administrativo. Las magnitudes que fluctúan de continuo se publican solo cuando superan una banda muerta, para no inundar la red con ruido de medición ni agotar la cuota del plan gratuito; el estado de la red, en cambio, se publica en cada transición sin banda muerta, porque es el dato crítico y ningún cambio puede perderse. Los acumulados viajan cada treinta segundos porque no son urgentes.

Las tres últimas variables son de lectura y escritura, y cierran el lazo de control: desde la aplicación `Arduino IoT Cloud Remote` del teléfono el usuario puede silenciar la alarma y ajustar los umbrales de protección sin tocar el tablero eléctrico. El procedimiento completo de alta del dispositivo, creación del Thing y montaje del tablero está documentado paso a paso en el archivo `GUIA_ARDUINO_CLOUD.md` que acompaña al código.

6. Revisión del diseño a partir de la retroalimentación

El facilitador planteó tres observaciones sobre la primera versión del sistema: si la instalación es de una o de dos fases, si lo que se protege es el inversor o los electrodomésticos, y la necesidad de una batería para que el equipo no deje de vigilar el voltaje mientras el programa está ocupado reconectando. Las tres son correctas y la tercera señala un defecto de fondo que el diseño original no resolvía. Esta sección documenta el análisis y los cambios aplicados en la versión 2.0.

6.1 ¿Una fase o dos fases?

La acometida residencial dominicana se presenta en dos configuraciones. La monofásica a dos hilos entrega 120 V entre una línea viva y el neutro, y es la de apartamentos y viviendas pequeñas sin equipos de 240 V. La monofásica trifilar, llamada corrientemente fase partida, entrega dos líneas vivas desfasadas 180 grados más un neutro: 120 V de cada línea al neutro y 240 V entre ambas. Es la habitual en viviendas con aire acondicionado, calentador eléctrico, estufa o bomba de agua.

La primera versión medía una sola línea, y esa simplificación deja ciegas dos fallas que son precisamente las más caras.

Fase perdida

Un fusible del transformador de la distribuidora puede abrirse y dejar sin servicio una sola de las dos líneas. Si el monitor vigila L1 y la que muere es L2, el sistema no detecta absolutamente nada: reporta 120 V correctos mientras media vivienda queda sin luz. Peor aún, los equipos de 240 V quedan alimentados con la mitad de la tensión, que es la forma más rápida de quemar el compresor de un aire acondicionado.

Neutro perdido

Es la falla más destructiva de una instalación residencial y merece una explicación, porque no es intuitiva. Si el conductor neutro se suelta, las dos líneas quedan conectadas en serie a través de las cargas de la casa. El voltaje de 240 V se reparte entonces entre ambas en proporción inversa a su carga: la línea con menos aparatos conectados sube hacia 180 o 200 V, y la otra colapsa. En cuestión de segundos se queman los equipos del lado que subió.

Con un solo sensor este fenómeno es literalmente invisible: si el sensor está en la línea que colapsó, el sistema cree que hay una subtensión ordinaria; si está en la que subió, cree que hay una sobretensión ordinaria. En ambos casos diagnostica mal. Con dos sensores la firma es inconfundible: la suma de las dos líneas se mantiene cerca de 240 V mientras la diferencia entre ellas se dispara. Por eso la versión 2.0 declara el estado NEUTRO_PERDIDO cuando el desbalance supera los 25 V, y lo evalúa antes que cualquier otra condición.

La decisión de diseño fue implementar el caso general de fase partida, con dos sensores de voltaje y dos de corriente. Para una vivienda monofásica simple basta con poner la constante MONOFASICO en true: el mismo programa ignora entonces la segunda línea y desactiva la detección de desbalance, sin tocar una sola línea de lógica.

6.2 ¿Se protege el inversor o se protegen los electrodomésticos?

La pregunta del facilitador expone algo que el diseño original daba por supuesto sin decirlo. Son dos objetivos distintos y en ciertos momentos opuestos, y conviene resolverlos explícitamente.

Tabla 7. Los tres escenarios de protección y cómo los resuelve el sistema.

Escenario	Qué se protege	Cómo actúa el sistema
A. Con inversor	Los electrodomésticos	Ante voltaje peligroso transfiere la carga al inversor. Los equipos quedan alimentados con onda limpia. Es el objetivo principal del proyecto.
B. Con inversor	El inversor y el banco de baterías	Aquí aparece el conflicto: el inversor aguanta 2400 W y la vivienda puede pedir 5000 W. Se activa el deslastre de carga.
C. Sin inversor	Los electrodomésticos, sin continuidad	No hay adónde transferir, pero sí de qué proteger: abre el contactor, desconecta la casa de la red peligrosa, avisa, y reconecta al normalizarse.

El conflicto del escenario B es real y frecuente. Transferir toda la vivienda a un inversor de menor capacidad lo sobrecarga y lo hace apagarse por protección, con lo que se pierde incluso el refrigerador, que era justamente lo que se quería salvar. La respuesta es el deslastre de carga: un segundo relevador corta los circuitos no esenciales —aire acondicionado, calentador, secadora— y deja vivos los esenciales, que son la nevera, la iluminación y las comunicaciones.

El deslastre se dispara por dos motivos independientes. El primero es que el consumo medido supere la capacidad del inversor. El segundo es que el banco de baterías caiga por debajo de 11.8 V, porque descargar una batería de plomo-ácido más allá del cincuenta por ciento le acorta la vida útil a la mitad; proteger la batería es proteger la inversión más cara de la instalación.

6.3 La alimentación del propio monitor

Esta es la observación más importante de las tres, porque señala un defecto que ninguna cantidad de buena programación podía corregir. Conviene separar dos problemas que se parecen y que tienen soluciones completamente distintas.

Tabla 8. Dos problemas distintos que impedían vigilar el voltaje.

Problema	Por qué ocurre	Solución
De software	Una tarea lenta —reconectar el WiFi puede tardar 8 s— bloquea a la tarea crítica que vigila el voltaje.	FreeRTOS. La tarea de red vive en el núcleo 0 y puede bloquearse sin que el muestreo, en el núcleo 1 y con prioridad superior, pierda un solo ciclo.
De hardware	En un apagón el equipo se queda sin corriente y deja de existir.	Alimentación de respaldo. Ninguna arquitectura de software salva a un microcontrolador apagado.

La versión 1 resolvía el primero y ni siquiera se planteaba el segundo. Alimentaba el ESP32 desde los 120 V AC mediante una fuente HLK-PM01, de modo que el equipo se apagaba en el instante exacto del apagón: no podía cronometrar el corte, ni contarle, ni avisar de él. Un monitor de apagones que se apaga con el apagón es una contradicción.

La corrección aplicada consiste en alimentar el ESP32 desde el bus de 12 V del banco de baterías del inversor, mediante un convertidor reductor MP1584 o LM2596 a 5 V. La elección tiene tres ventajas sobre añadir una batería independiente: el monitor permanece activo durante todo el apagón, que es cuando sus datos valen algo; no hay una segunda batería que mantener y reemplazar; y de paso el equipo puede medir el voltaje del banco, estimar la autonomía restante y avisar antes de que el inversor se apague solo.

Para leer los 12 V con el ADC de 3.3 V del ESP32 se añade un divisor de tensión de 100 k y 33 k. Con esa medida el sistema estima el estado de carga y genera dos avisos escalonados: a 11.8 V activa el deslastre, y a 11.4 V emite una alarma de tono agudo porque el inversor está a punto de desconectarse. En una vivienda sin inversor, la alternativa equivalente es una celda 18650 con módulo de carga TP4056 y elevador a 5 V.

El presupuesto de la sección 8 incorpora estos componentes adicionales: el segundo juego de sensores de voltaje y corriente, el relevador de deslastre y el convertidor reductor con su divisor de tensión.

7. Evidencias demostrables de programación concurrente

Esta sección responde al tercer componente de la asignación. Se desarrollaron seis programas ejecutables, ordenados como una progresión argumental: el primero muestra el problema, los cuatro siguientes construyen la solución capa por capa, y el último la mide en una PC. Todos son reproducibles desde los enlaces del Anexo A.

Tabla 9. Resumen de las seis demostraciones desarrolladas.

#	Demostración	Plataforma	Qué prueba
1	Secuencial bloqueante	Arduino UNO	Que delay() congela el sistema completo y hace que se pierdan eventos externos.
2	Multitarea cooperativa	Arduino UNO	Que hay concurrencia real sobre un solo núcleo, sin sistema operativo, usando millis().
3	Hilos FreeRTOS	ESP32	Expulsión de tareas por prioridad y paralelismo verdadero sobre dos núcleos.
4	Carrera y mutex	ESP32	Que la actualización perdida es real, y que el mutex la elimina por completo.
5	Nodos MQTT	ESP32 + Python	Programación distribuida: mensajes, tolerancia a fallo parcial y órdenes remotas.
6	Hilos vs. procesos	Python 3.11	Con cronómetro: cuándo sirven los hilos, cuándo hacen falta procesos y por qué.

7.1 Ejemplo 1 — El problema: programación secuencial bloqueante

Se pide al programa hacer tres cosas: parpadear un LED cada segundo, emitir un pitido cada 300 ms y responder a un botón de emergencia. Escrito con delay(), el microcontrolador queda detenido dentro de cada llamada y no ejecuta absolutamente nada más, ni siquiera mirar el botón.

El resultado es doble y ambas fallas son graves. Primero, los períodos se contaminan: los delay() se suman y el ciclo que debía durar 1000 ms dura en realidad unos 1300 ms. Segundo, y más serio, el botón solo se consulta durante una fracción minúscula de cada ciclo, de modo que un toque corto se ignora casi siempre. En un sistema de protección eléctrica, ese botón podría ser el paro de emergencia.

7.2 Ejemplo 2 — Multitarea cooperativa con millis()

Las mismas tres obligaciones, reescritas sin una sola llamada bloqueante. Cada tarea se convierte en una máquina de estados que consulta el reloj, se pregunta «¿ya me toca?», hace su trabajo en microsegundos y devuelve el control. El loop() deja de ser un procedimiento y pasa a ser un planificador cooperativo.

El botón ahora responde al instante, y los períodos del LED y del zumbador se mantienen exactos e independientes entre sí. Esto es concurrencia real sobre un solo núcleo: las tres tareas progresan intercaladas, aunque en cada instante físico solo se ejecute una instrucción.

Es importante señalar también su límite, porque es lo que motiva el ejemplo siguiente: el modelo se llama cooperativo porque nadie puede expulsar a una tarea. Si una sola se porta mal y tarda dos

segundos, congela a todas las demás. La cooperación depende de la disciplina del programador, no del hardware.

7.3 Ejemplo 3 — Hilos reales, expulsión y dos núcleos

Este ejemplo demuestra por separado las dos ideas que suelen confundirse. Para la primera se crean dos tareas en el mismo núcleo: una de tiempo real que parpadea un LED con período exacto (prioridad 3) y otra escrita a propósito para portarse mal (prioridad 1), que bloquea tres segundos y luego quema CPU durante dos más. En el modelo cooperativo del Ejemplo 2 eso habría congelado todo el sistema; aquí el LED no se altera ni un milisegundo, porque el planificador expulsa a la tarea trágica sin pedirle permiso.

Para la segunda idea se crean dos tareas gemelas, ancladas una a cada núcleo físico, que cuentan lo más rápido que pueden durante un segundo e imprimen su resultado junto con `xPortGetCoreID()`. Que ambos contadores avancen simultáneamente es la prueba de que esto ya no es «turnarse muy rápido»: es ejecutarse a la vez. Eso es paralelismo.

7.4 Ejemplo 4 — La condición de carrera y su cura

El error más caro de la programación concurrente no es que el programa se caiga: es que devuelva un número equivocado sin avisar. Este sketch lo reproduce y lo corrige en la misma ejecución, para poder comparar.

Dos tareas, una en cada núcleo, acumulan energía en un mismo totalizador compartido: cada una suma 1.0 Wh, mil veces. El resultado correcto es evidentemente 2000.0 Wh. En la fase 1, sin protección, el total impreso es sistemáticamente menor y cambia en cada ejecución. Esa inestabilidad es la firma inconfundible de una condición de carrera. En la fase 2, con el mutex, el total es exactamente 2000.00 siempre.

El mismo código. Los mismos datos. La misma máquina. Lo único que cambia es el candado.

7.5 Ejemplo 5 — Programación distribuida con MQTT

Aquí el trabajo se reparte entre máquinas distintas, que no comparten memoria ni reloj y se comunican solo por mensajes. Tres nodos ESP32 —tablero principal, aire acondicionado y bomba de agua— publican su consumo; un concentrador escrito en Python se suscribe a todos, suma el consumo del hogar y, si supera la capacidad del inversor, emite la orden de deslastre de carga que los nodos obedecen.

Los tres problemas clásicos de lo distribuido se atacan explícitamente: el testamento de MQTT (LWT) hace que el broker anuncie automáticamente la muerte de un nodo; los números de secuencia por nodo permiten descartar mensajes que llegan fuera de orden; y los mensajes retenidos garantizan que un suscriptor nuevo conozca de inmediato el último estado de cada nodo.

El concentrador, además, es un segundo ejemplo de concurrencia en un lenguaje distinto: usa tres hilos —red, supervisión y presentación— sobre un diccionario compartido protegido con `threading.Lock`. Es exactamente el problema del Ejemplo 4, con otro vocabulario.

7.6 Ejemplo 6 — Concurrencia y paralelismo, medidos con cronómetro

Los ejemplos anteriores demuestran los conceptos sobre un microcontrolador. Este los mide en una PC, de modo que la conclusión no es una opinión sino segundos. Se ejecutó sobre Python 3.11 en un equipo de 12 núcleos, y los resultados fueron los siguientes:

Tabla 10. Resultados medidos del Ejemplo 6 (Python 3.11, 12 núcleos).

Prueba	Secuencial	Resultado	Interpretación
8 lecturas remotas — con 8 hilos	4.00 s	0.51 s (7.87×)	El cuello de botella era la ESPERA: ocho tareas caben en el tiempo de una.
8 lecturas remotas — con asyncio	4.00 s	0.51 s (7.87×)	Mismo resultado sin crear un solo hilo: la concurrencia no necesita hilos.
8 cálculos de RMS — con 8 hilos	5.21 s	5.27 s (0.99×)	Ninguna mejora. El GIL solo deja ejecutar bytecode a un hilo a la vez.
8 cálculos de RMS — con procesos	5.21 s	1.82 s (2.86×)	Paralelismo real: cada proceso es un intérprete sobre un núcleo distinto.

La lectura de esta tabla es la conclusión práctica más útil de todo el trabajo: no existe «la forma rápida» de programar concurrente. La herramienta correcta depende de dónde se pierde el tiempo. Si la tarea espera —red, disco, un sensor—, los hilos o asyncio ganan, porque mientras uno espera los demás avanzan. Si la tarea calcula, en CPython los hilos no ayudan en absoluto y hay que recurrir a procesos.

Aplicado a SmartPower RD: consultar diez sensores por la red es un problema de espera y se resuelve con concurrencia; reentrenar el modelo que predice el consumo es un problema de CPU y exige paralelismo real.

La tercera parte del programa reproduce la condición de carrera del Ejemplo 4 en Python, con cuatro hilos sumando sobre un diccionario compartido. El resultado medido fue contundente:

```
PARTE 3 - Condicion de carrera y Lock (el Ejemplo 4, en Python)
=====
Esperado           : 200,000
Sin Lock (4 hilos)  : 50,126   -> perdidas: 149,874 (74.9 %)
Con Lock (4 hilos)  : 200,000 -> perdidas: 0      ( 0.0 %)
```

Figura 4. Salida real del programa concurrencia_comparada.py. Se perdió el 74.9 % de las operaciones por falta de un candado.

Tres de cada cuatro operaciones desaparecieron sin que el programa emitiera ni un solo error. Ningún compilador advierte de esto, ninguna excepción se lanza y ninguna prueba unitaria escrita de la forma habitual lo detecta. Por eso los defectos de concurrencia no se corrigen «probando»: se corrigen protegiendo el dato.

8. Presupuesto y cotización

El presupuesto se presenta en dos escenarios, porque responden a preguntas distintas: el primero es lo que cuesta demostrar el proyecto en el marco de esta asignatura; el segundo es lo que costaría instalarlo realmente en una vivienda.

Todos los montos en RD\$ se calcularon con la tasa de venta del dólar estadounidense publicada por el Banco Central de la República Dominicana para el 5 de agosto de 2026: RD\$58.4424 por US\$1.00. Los precios unitarios son valores de referencia del mercado local e internacional a esa fecha y deben confirmarse mediante cotización formal con el suplidor antes de ejecutar la compra.

8.1 Escenario A — Prototipo académico simulado

El caso práctico de esta asignatura se desarrolló íntegramente sobre el simulador Wokwi y el entorno de Arduino Cloud, sin adquirir un solo componente físico. Esta decisión no es solo económica: permite que el facilitador reproduzca cada demostración desde su propio navegador, sin depender de que el hardware del estudiante funcione el día de la evaluación.

Tabla 11. Escenario A: costo del prototipo académico.

Concepto	Detalle	US\$	RD\$
Simulador Wokwi	Plan gratuito	0.00	0.00
Arduino Cloud	Plan Free (hasta 2 Things)	0.00	0.00
Broker MQTT de pruebas	test.mosquitto.org, público	0.00	0.00
Arduino IDE / VS Code	Software libre	0.00	0.00
Python 3.11 y bibliotecas	Software libre	0.00	0.00
Horas de desarrollo	38 h de trabajo propio del estudiante	0.00	0.00
TOTAL ESCENARIO A		0.00	0.00

8.2 Escenario B — Implementación física en una vivienda

8.2.1 Materiales y componentes

Tabla 12. Escenario B: cotización de materiales.

Componente	Cant.	P. unit. US\$	Subtotal US\$	Subtotal RD\$
ESP32 DevKit V1 (WiFi, doble núcleo)	1	8.00	8.00	467.54
Sensor de voltaje AC ZMPT101B (una por línea)	2	6.00	12.00	701.31
Sensor de corriente SCT-013-030 (30 A)	2	11.00	22.00	1,285.73
Módulo de relé / contactor 30 A	1	22.00	22.00	1,285.73
Pantalla LCD 16×2 con interfaz I2C	1	5.00	5.00	292.21
Fuente AC-DC Hi-Link HLK-PM01 (5 V, 3 W)	1	6.00	6.00	350.65

Componente	Cant.	P. unit. US\$	Subtotal US\$	Subtotal RD\$
Módulo RTC DS3231 + lector microSD	1	7.50	7.50	438.32
Gabinete plástico IP54 con riel DIN	1	18.00	18.00	1,051.96
Protecciones (fusible, varistor MOV, borneras)	1	9.00	9.00	525.98
Zumbador, LED, botón, resistencias, cableado	1	6.00	6.00	350.65
Fabricación de PCB (2 unidades)	1	12.00	12.00	701.31
Breaker bipolar 30 A para transferencia	1	14.00	14.00	818.19
Relé de deslastre de carga 30 A	1	12.00	12.00	701.31
Convertidor reductor MP1584 + divisor de tensión	1	5.00	5.00	292.21
SUBTOTAL MATERIALES			158.50	9,263.12

8.2.2 Logística, impuestos y servicios

Tabla 13. Escenario B: costos adicionales y totales.

Concepto	Base	US\$	RD\$
Subtotal de materiales	—	158.50	9,263.12
Envío internacional y gestión aduanal	20 %	31.70	1,852.62
Subtotal con logística	—	190.20	11,115.74
ITBIS	18 %	34.24	2,001.07
TOTAL HARDWARE		224.44	13,116.81
Instalación por técnico electricista certificado	6 h	85.00	4,967.60
Pruebas, calibración y puesta en marcha	4 h	50.00	2,922.12
TOTAL INSTALADO (sin mano de obra de software)		359.44	21,006.53
Ingeniería de software (si se contrata a terceros)	38 h × US\$12	456.00	26,649.73
TOTAL DEL PROYECTO LLAVE EN MANO		815.44	47,656.26

En el Escenario A la ingeniería de software tiene costo cero porque la desarrolla el propio estudiante. Se incluye aquí valorizada para dimensionar el proyecto si se contratara externamente, y porque es el componente que se amortiza al replicar el sistema en más viviendas.

8.2.3 Costos recurrentes

Tabla 14. Costos operativos anuales.

Concepto	Frecuencia	US\$/año	RD\$/año
Arduino Cloud, plan Maker (opcional)	Mensual	83.88	4,902.15
Alternativa: VPS propio con Mosquitto + TLS	Mensual	60.00	3,506.54
Consumo eléctrico del equipo (2 W continuos)	Anual	1.20	70.13
Internet doméstico	—	Ya existente	0.00

Concepto	Frecuencia	US\$/año	RD\$/año
TOTAL RECURRENTE (opción VPS)		61.20	3,576.67

8.3 Análisis de retorno de la inversión

La justificación económica del sistema no está en el ahorro de energía, sino en el daño que evita. Un solo compresor de nevera dañado por subtensión sostenida cuesta en el mercado dominicano entre RD\$15,000 y RD\$22,000 entre pieza y mano de obra; el compresor de un aire acondicionado inverter supera con facilidad los RD\$35,000.

Tabla 15. Estimación conservadora del retorno de la inversión.

Concepto	Estimado anual RD\$	Base del cálculo
Daño evitado a electrodomésticos	9,000.00	Un evento de RD\$18,000 cada dos años
Ahorro por visibilidad del consumo	5,400.00	10 % sobre una factura de RD\$4,500/mes
Optimización del banco de baterías	3,000.00	Mayor vida útil por dimensionamiento correcto
BENEFICIO ANUAL ESTIMADO	17,400.00	
Inversión inicial (Escenario B, sin software)	21,006.53	
PERÍODO DE RECUPERACIÓN	14.5 meses	

Advertencia metodológica: estas cifras son estimaciones basadas en supuestos razonables, no en un estudio de campo. El ahorro del 10 % por visibilidad del consumo proviene de la literatura sobre retroalimentación energética en el hogar y varía enormemente según el comportamiento del usuario. El valor de esta tabla es dimensionar el orden de magnitud, no predecir un resultado.

9. Cronograma de implementación

El proyecto se planificó en ocho semanas, agrupadas en cuatro fases. Las columnas marcadas corresponden al alcance efectivamente cubierto en esta entrega académica.

Tabla 16. Cronograma de ejecución del proyecto.

Semana	Actividad	Entregable	Estado en esta entrega
1	Investigación del problema y estado del arte	Marco conceptual	Completado
2	Estudio de hilos, tareas y sincronización	Ejemplos 1 y 2	Completado
3	Arquitectura de tareas y diseño del sistema	Diagramas y tabla de tareas	Completado
4	Programación del firmware sobre FreeRTOS	SmartPowerRD.ino	Completado
5	Demostraciones de concurrencia y paralelismo	Ejemplos 3, 4 y 6	Completado
6	Topología distribuida y concentrador MQTT	Ejemplo 5 y gateway	Completado
7	Montaje físico y calibración de sensores	Prototipo en gabinete	Fase futura
8	Modelo de IA para predicción de consumo	Modelo entrenado	Fase futura

10. Conclusión

Este trabajo partió de una necesidad doméstica muy concreta —saber qué está pasando con la electricidad de mi casa y protegerla automáticamente— y terminó siendo, sobre todo, un ejercicio de programación concurrente. Esa transformación es, en sí misma, el hallazgo más valioso del proceso.

La conclusión técnica central es que la concurrencia no es una optimización que se añade al final, sino una decisión de arquitectura que se toma al principio. El Ejemplo 1 lo demuestra sin ambigüedad: un programa secuencial que usa `delay()` no es un programa lento que puede mejorarse ajustando constantes, es un programa que sencillamente no puede cumplir requisitos de tiempo real. Ninguna cantidad de optimización posterior lo arregla, porque el defecto está en el modelo de ejecución, no en el código.

La segunda conclusión es que concurrencia y paralelismo son cosas distintas, y confundirlas cuesta dinero medible. Los datos del Ejemplo 6 son inapelables: los mismos ocho hilos que multiplicaron por 7.87 el rendimiento de un trabajo dominado por la espera produjeron una aceleración de $0.99\times$ —es decir, ninguna— en un trabajo dominado por la CPU. La herramienta correcta no depende de cuál sea «la mejor», sino de dónde se pierde el tiempo.

La tercera conclusión, y la que más me sorprendió al desarrollar el trabajo, tiene que ver con la naturaleza de los errores de concurrencia. Cuando el programa del Ejemplo 6 perdió el 74.9 % de sus

operaciones, no se cayó, no lanzó ninguna excepción y no emitió ni una advertencia: simplemente devolvió un número equivocado con absoluta serenidad. En un sistema que factura energía, ese silencio es el peor comportamiento posible. Los defectos de concurrencia no se descubren probando; se evitan protegiendo el dato desde el diseño.

En cuanto al salto hacia lo distribuido, el Ejemplo 5 dejó claro que agregar nodos no es simplemente «más de lo mismo». Al cruzar la frontera de la red se pierden tres garantías que en un solo chip se daban por sentadas —memoria común, reloj común y ausencia de fallo parcial— y cada una obliga a un mecanismo específico: números de secuencia, mensajes retenidos y testamento. Diseñar para el fallo deja de ser una buena práctica y pasa a ser un requisito.

Desde el punto de vista de viabilidad, el análisis económico sitúa la implementación física en aproximadamente RD\$21,007 instalada, con un período de recuperación estimado en catorce meses y medio fundamentado principalmente en el daño evitado a los electrodomésticos. Es una inversión razonable para el contexto dominicano, y su costo marginal disminuye al replicarse porque la ingeniería de software se amortiza entre todas las unidades.

Finalmente, y en relación con la asignatura, este proyecto muestra con claridad por qué la inteligencia artificial y el Internet de las Cosas se estudian juntos. El IoT es lo que genera el dato: sin sensores que midan el voltaje cada cincuenta milisegundos no hay serie temporal que analizar. La IA es lo que convierte ese dato en una decisión útil: predecir la autonomía de la batería, identificar qué electrodoméstico se encendió o anticipar la falla de un compresor semanas antes de que ocurra. Y en el medio, sosteniendo ambas cosas, está la programación concurrente, que es lo que permite que un chip que cuesta menos de quinientos pesos haga cinco cosas a la vez sin equivocarse en ninguna.

10.1 Recomendaciones

- Antes de conectar el prototipo a 120 V AC reales, validar el aislamiento del ZMPT101B y trabajar siempre con la instalación desenergizada. La simulación no protege de un error de cableado.
- Sustituir el broker público test.mosquitto.org por un broker propio con TLS y autenticación antes de cualquier uso real: un topic público expone el patrón de ocupación de la vivienda, que es información sensible.
- Usar la tarea de diagnóstico para dimensionar las pilas con el dato del high water mark en lugar de estimarlas. Un desbordamiento de pila en FreeRTOS se manifiesta como un reinicio aparentemente aleatorio, muy difícil de diagnosticar después.
- No usar portMAX_DELAY al tomar un mutex en código de producción. Un vencimiento explícito convierte un interbloqueo en una degradación del servicio, que es infinitamente preferible.
- Incorporar en la siguiente fase el registro local en microSD, para que el histórico sobreviva a una caída prolongada de internet, que es exactamente cuando más valor tiene el dato.

11. Fuentes de investigación y bibliografía

11.1 Libros y artículos académicos

Barry, R. (2016). Mastering the FreeRTOS Real Time Kernel: A Hands-On Tutorial Guide. Real Time Engineers Ltd.

Lee, E. A. (2006). The Problem with Threads. Computer, 39(5), 33-42. IEEE Computer Society.

Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). Operating System Concepts (10.^a ed.). John Wiley & Sons.

Tanenbaum, A. S., & Bos, H. (2015). Modern Operating Systems (4.^a ed.). Pearson Education.

Tanenbaum, A. S., & Van Steen, M. (2017). Distributed Systems: Principles and Paradigms (3.^a ed.). Pearson.

11.2 Documentación técnica y normas

Arduino. (2026). Arduino Cloud Documentation. <https://docs.arduino.cc/arduino-cloud/>

Espressif Systems. (2026). ESP-IDF Programming Guide: FreeRTOS (IDF) API Reference. <https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/system/freertos.html>

Espressif Systems. (2026). ESP32 Series Datasheet y Technical Reference Manual. <https://www.espressif.com/en/support/documents/technical-documents>

OASIS. (2019). MQTT Version 5.0 — OASIS Standard. <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>

Python Software Foundation. (2026). Documentación de las bibliotecas threading, multiprocessing y asyncio, Python 3.11. <https://docs.python.org/3/library/concurrency.html>

Wokwi. (2026). Wokwi Documentation: ESP32 Simulator. <https://docs.wokwi.com/>

11.3 Conferencias y recursos en línea

Pike, R. (2012). Concurrency is not Parallelism. Heroku Waza Conference. <https://go.dev/blog/waza-talk>

Red Hat. (2023). IoT: ¿qué es el Internet de las cosas y cómo funciona? <https://www.redhat.com/es/topics/internet-of-things/what-is-iot>

Jones, M. B. (1997). What Really Happened on Mars Rover Pathfinder. The Risks Digest, 19(49).

11.4 Fuentes de datos económicos

Banco Central de la República Dominicana. (2026). Tasa de cambio del dólar estadounidense, 5 de agosto de 2026: RD\$58.0760 (compra) / RD\$58.4424 (venta). <https://www.bancentral.gov.do/>

Anexo A. Enlaces de acceso al código y a los videos

La asignación exige que las evidencias sean accesibles desde una URL para su validación. Este anexo separa las evidencias en dos grupos: las de elaboración propia (A.1) y el material audiovisual de terceros que la propia asignación admite (A.2).

A.1 Código fuente

Todo el código de este proyecto es de elaboración propia y está publicado en un repositorio público de GitHub, accesible sin necesidad de iniciar sesión:

<https://github.com/Criscarr26/smartpower-rd>

El repositorio contiene el firmware del proyecto principal y las seis demostraciones de la sección 6, cada una con su montaje listo para el simulador. Su estructura es la siguiente:

smartpower-rd/	
— README.md	guía de ejecución
— SmartPowerRD/	proyecto principal (ESP32)
— SmartPowerRD.ino	5 tareas de FreeRTOS
— diagram.json	montaje para el simulador
— libraries.txt	dependencias
— SmartPowerRD_ArduinoCloud/	version para Arduino Cloud
— SmartPowerRD_ArduinoCloud.ino	misma logica, tarea de nube
— thingProperties.h	las 10 variables de nube
— arduino_secrets.h	Device ID y Secret Key
— GUIA_ARDUINO_CLOUD.md	alta del Thing y del tablero
— ejemplos_concurrencia/	las seis demostraciones
— 01_secuencial_bloqueante/	el problema: delay()
— 02_multitarea_cooperativa/	concurrencia con millis()
— 03_hilos_freertos/	expulsión y dos núcleos
— 04_condicion_de_carrera_mutex/	la carrera y su corrección
— 05_distribuido_mqtt/	nodo ESP32 + gateway Python
— 06_python_hilos_vs_procesos/	medición con cronómetro
— concurrencia_comparada.py	
— salida_de_la_corrida.txt	evidencia de la ejecución

El montaje del proyecto principal fue validado contra el simulador: los nombres de pines corresponden a la placa board-esp32-devkit-c-v4 y las trece piezas del diagram.json se instancian sin error.

A.2 Material audiovisual de referencia

La asignación admite expresamente que las evidencias en video provengan «de otros ejemplos o desarrollos». Se seleccionaron los siguientes videos públicos, uno o varios por cada concepto demostrado en la sección 6, priorizando material en español de canales técnicos reconocidos. Cada enlace fue comprobado individualmente: se verificó que el video existe, que es de acceso público y que su contenido corresponde al tema indicado.

Estos videos son material de terceros y se citan únicamente como apoyo conceptual. La autoría corresponde a cada canal indicado. El código de este trabajo es de elaboración propia y no procede de estas fuentes.

Tabla 17. Videos de referencia por concepto demostrado.

#	Concepto	Video de referencia y enlace de acceso
—	Proyecto principal	«Medidor Energético Inteligente 2.0 con Arduino ESP32 + Raspberry Pi + Home Assistant» MCMCHRIS (Christopher Méndez) · español https://www.youtube.com/watch?v=efSfnv4YqtY
1	delay() bloqueante	«Arduino desde cero en Español – Capítulo 80: Código sin bloqueo y librería TimeAlarms» bitwiseAr · español https://www.youtube.com/watch?v=tdFsSo5LVhg
1	delay() bloqueante	«MULTITAREA (multitasking) parte #1: función MILLIS vs DELAY en ARDUINO» Nuevas Tecnologías JA · español https://www.youtube.com/watch?v=MOmMq6zmv24
1	delay() bloqueante	«Arduino Sketch with millis() instead of delay()» Programming Electronics Academy · inglés https://www.youtube.com/watch?v=BYKQ9rk0FEQ
2	Multitarea cooperativa	«¿Cómo hacer MULTITAREA en ARDUINO? Ejecuta múltiples tareas con la función millis()» David Portilla – Programación y Electrónica · español https://www.youtube.com/watch?v=ikQ7h1kaSr8
2	Multitarea cooperativa	«MÁQUINA de ESTADOS FINITOS con ARDUINO / [MULTITAREAS] #16» Nuevas Tecnologías JA · español https://www.youtube.com/watch?v=iPepH8Co4YI
2	Multitarea cooperativa	«Explicación de la función millis para multitareas con ejemplo» Aprendamos Ingeniería · español https://www.youtube.com/watch?v=oGTwphcGZu8
3	Hilos y dos núcleos	«Selección del núcleo en el que corren las tareas: freeRTOS, xTaskCreatePinnedToCore, ESP32» Vladimir Trujillo · español https://www.youtube.com/watch?v=vZGikmb45Y4
3	Hilos y dos núcleos	«Ejemplo PRÁCTICO con FreeRTOS y ESP32: Tareas (vTaskCreate, vTaskDelay, vTaskDelete)» Fer Daniele · español https://www.youtube.com/watch?v=LA9N-3h5fIU
3	Hilos y dos núcleos	«ESP32 IDF SDK 11: FreeRTOS TASK Dual Core» Easy Learning · español https://www.youtube.com/watch?v=J3itE-oXuco
4	Carrera y mutex	«MUTEX en FreeRTOS (ESP32): domina la exclusión mutua entre tareas (ejemplo práctico)» Fer Daniele · español https://www.youtube.com/watch?v=2XVyRA92dzw
4	Carrera y mutex	«Condiciones de Carrera – Race Conditions en Concurrency» Facundo Carballo · español https://www.youtube.com/watch?v=pG-72z0H_dQ
4	Carrera y mutex	«Curso freeRTOS & ESP32: Video 4 (Mutex)» biblioman09 · español https://www.youtube.com/watch?v=Zy8mKkAtEJw
4	Carrera y mutex	«Introduction to RTOS Part 6 – Mutex» Digi-Key Electronics (Shawn Hymel) · inglés https://www.youtube.com/watch?v=I5SauRpbITs
5	MQTT distribuido	«Protocolo MQTT (ESP32 + Broker Gratuito)» Ignacio Aguilera · español https://www.youtube.com/watch?v=x5GML1FqcTQ
5	MQTT distribuido	«Múltiples ESP32 WiFi (Broker Gratuito MQTT)» Ignacio Aguilera · español https://www.youtube.com/watch?v=Ik4qCZiVcxU

#	Concepto	Video de referencia y enlace de acceso
6	Hilos, procesos y GIL	«PyConAr 2021 – Concurrencia y paralelismo en Python: Multithreading vs Multiprocessing vs Async» PyAr – Python Argentina · español https://www.youtube.com/watch?v=u77Az26bFPA
6	Hilos, procesos y GIL	«Threading, MultiProcessing y AsyncIO en Python [TEORÍA]» Dimas · español https://www.youtube.com/watch?v=UWJWDOo81Xc
6	Hilos, procesos y GIL	«Python Threading Tutorial: Run Code Concurrently Using the Threading Module» Corey Schafer · inglés https://www.youtube.com/watch?v=IEEhzQoKtQU
6	Hilos, procesos y GIL	«The Python Global Interpreter Lock – Explained» Tech With Tim · inglés https://www.youtube.com/watch?v=XVcRQ6T9RH0
*	Simulador Wokwi	«Curso IoT con ESP32 y Arduino (N.º 3): simulador online de ESP32 y Arduino – Wokwi» Editronikx · español https://www.youtube.com/watch?v=DtORRxxbyMQ
*	Simulador Wokwi	«Instalar y usar Wokwi paso a paso: simula circuitos con Arduino (guía para principiantes)» DASSLOGIC · español https://www.youtube.com/watch?v=SGk0kPFPKRI
*	Simulador Wokwi	«WOKWI – Simular ESP32, Arduino y Raspberry Pi Pico» Jadsa Tech · español https://www.youtube.com/watch?v=Emm2L8CTSPA

Enlaces comprobados el 7 de agosto de 2026. Si alguno dejara de estar disponible, el mismo contenido puede localizarse buscando el título exacto entre comillas junto al nombre del canal.

A.3 Qué se entrega

La entrega en la plataforma de la asignatura consta de dos piezas, siguiendo el mismo formato de las entregas anteriores de la materia:

- **Este documento en PDF.** Contiene la investigación, el desarrollo del caso práctico, el presupuesto y, en este anexo, todos los enlaces de validación.
- **La carpeta SmartPowerRD con el sketch de Arduino.** Es el código del proyecto principal, el mismo que está publicado en el repositorio junto con las seis demostraciones.

Los videos y el código completo no se adjuntan como archivos: se consultan desde el enlace del repositorio y desde la tabla 15 de este anexo, tal como pide el enunciado.

Cómo reproducir cada demostración

- Abrir wokwi.com, crear un proyecto nuevo del tipo indicado (Arduino UNO para los ejemplos 1 y 2; ESP32 para los ejemplos 3, 4, 5 y el proyecto principal).
- Pegar el contenido del archivo .ino en la pestaña del sketch y el diagram.json de esa misma carpeta en la pestaña del mismo nombre. Cada ejemplo trae su montaje ya cableado, así que no hay que colocar piezas a mano.

- Solo en el proyecto principal: al pulsar «Start simulation» por primera vez, Wokwi ofrecerá instalar la librería LiquidCrystal I2C; hay que aceptarla y añadir después la línea PubSubClient en la pestaña libraries.txt.
- Pulsar «Start simulation» y abrir el monitor serie. En el proyecto principal, mover el potenciómetro del voltaje para provocar el apagón y observar la transferencia al inversor.
- Para el ejemplo 6, ejecutar en la PC: `python concurrencia_comparada.py`. No requiere instalar dependencias.
- Para el ejemplo 5, ejecutar `pip install paho-mqtt` y luego `python gateway_concentrador.py` mientras la simulación del nodo está corriendo.